

University
Politehnica
of Bucharest





PL/SQL (II)

Ș.L. Dr. Ing. Ciprian-Octavian Truică
ciprian.truica@cs.pub.ro



Overview

- Variables and constants
- Data types
- Operators
- Control statements



Overview

- Variables and constants
- Data types
- Operators
- Control statements



Variables and constants

- A PL/SQL can contain
  - Variables
  - Constants
  - External variables



Variables and constants

- Variables can be
  - Scalar
    - Contain a single value
    - No internal components
  - Composit
    - Collections
    - Records



Declaring Variables and constants

- When declaring a variable
  - Storage space is allocated for a value of a specified data type
  - The storage location is marked with a name so it can be referenced
- Objects must be declared before referencing them
- Declarations can appear in the declarative part of any
  - Block
  - Subprogram
  - Package

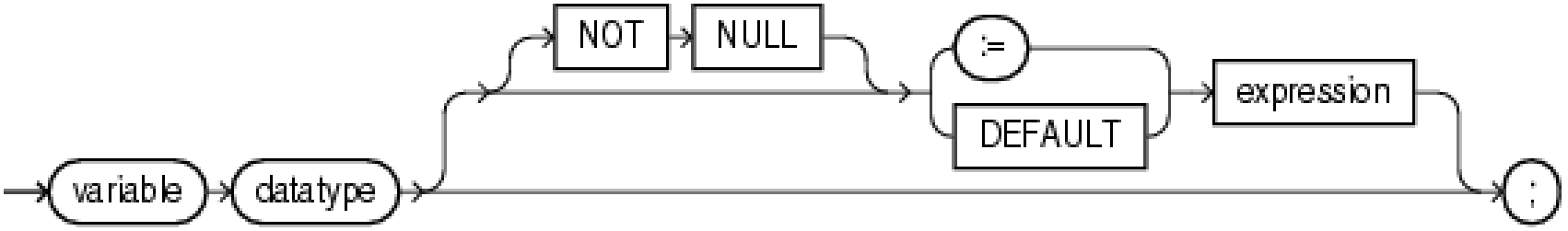


Declaring Variables

- A variable declaration always specifies
  - The name
  - Data type
- A variable declaration can also specify an initial value for most data types
- The variable name must be a valid user-defined identifier
- The data type can be any PL/SQL data type

Declaring Variables

- The PL/SQL data types include SQL data types
- A data type is either
  - Scalar (without internal components)
  - Composite (with internal components)
- Syntax for declaring a variable



variable [CONSTANT] datatype [{ NOT NULL {:= | DEFAULT} expression }] ;

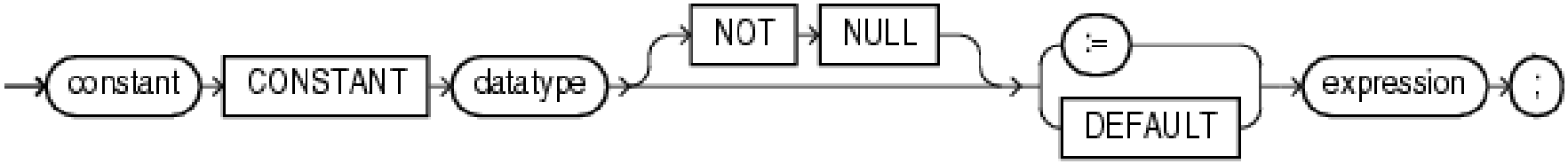


Declaring Constants Syntax

- **variable** - Name of the variable that you are declaring
- **datatype** - Name of a scalar data type, including any qualifiers for size, precision, and character or byte semantics
- **NOT NULL** - Imposes the NOT NULL constraint on the variable
- **expression** - Value to be assigned to the variable when the declaration is elaborated.
- *The **expression** and **variable** must have compatible data types.*

Declaring Constants

- A constant holds a value that does not change
- A constant declaration has two more requirements:
  - The keyword CONSTANT
  - The initial value of the constant.
- The initial value of a constant is its permanent value
- Syntax for declaring a constant



constant CONSTANT datatype [NOT NULL] { := | DEFAULT } expression ;



Initial Values of Variables and Constants

- **constant** - Name of the constant that you are declaring
- **datatype** - Data type for which a variable can be declared with an initial value
- **NOT NULL** - Imposes the NOT NULL constraint on the constant.
- **expression** - Initial value for the constant. expression must have a data type that is compatible with datatype.
- *When constant declaration is elaborated, the value of expression is assigned to constant.*



NOT NULL Constraint

- A NOT NULL constraint can be imposed on
  - Scalar variables
  - Constants
  - Scalar components of
    - Composite variables
    - Constants
- The NOT NULL constraint prevents assigning a null value to the item
- To test for a NULL value, use the IS [NOT] NULL operator



NOT NULL Constraint

- An item can acquire this constraint either
  - Implicitly (from its data type)
  - Explicitly
- The default initial value for a scalar variable is NULL
- A scalar variable declaration that specifies NOT NULL (either implicitly or explicitly) must assign an initial value to the variable
- PL/SQL treats any zero-length string as a NULL value, this includes
  - Values returned by character functions
  - BOOLEAN expressions.



Initial Values of Variables and Constants

- If the declaration is
  - A variable: the initial value is optional unless the NOT NULL constraint is specified
  - A constant: the initial value is required

- If the declaration is in a block or subprogram:
  - The initial value is assigned to the variable or constant every time control passes to the block or subprogram

- If the declaration is in a package specification
  - The initial value is assigned to the variable or constant for each session (whether the variable or constant is public or private)



Initial Values of Variables and Constants

- To specify the initial value use
  - The assignment operator (:=) followed by an expression
  - The keyword DEFAULT followed by an expression
- The expression can include previously declared
  - Constants
  - Initialized variables
  - Functions results
- If no initial value is specified for a variable, then a value must be assigned before using it in any other context.



Declaring Items using %TYPE

- The %TYPE attribute permits the declaration of a data item of the same data type as a previously declared variable or column (without knowing what that type is)
- If the declaration of the referenced item changes, then the declaration of the referencing item changes accordingly
- The syntax of the declaration is:

referencing_item referenced_item%TYPE;



Declaring Items using %TYPE

- The referencing item inherits the following from the referenced item:
  - Data type and size
  - Constraints (unless the referenced item is a column)
- The referencing item does not inherit the initial value of the referenced item:
  - If the referencing item specifies or inherits the NOT NULL constraint, an initial value for it must be specified
- The %TYPE attribute is particularly useful when declaring variables to hold database values
- The syntax for declaring a variable of the same type as a column is:

variable_name table_name.column_name%TYPE;



Overview

- Variables and constants
- Data types
- Operators
- Control statements



Data types

- Every PL/SQL constant, variable, parameter, and function return value has a data type

- The data type determines
  - The storage format
  - Valid values
  - Operations



Scalar data types

- Scalar data types store values with no internal components
- A scalar data type can have subtypes
- A subtype is a data type that is a subset of another data type (its base type)
- A subtype has the same valid operations as its base type
- A data type and its subtypes comprise a data type family



Scalar data types

- PL/SQL predefines many types and subtypes
- The predefined data types are available in the STANDARD package
- Developers can also define their own subtypes (UDST – User Defined Subtypes)



Scalar data types

- The PL/SQL scalar data types are:
  1. The SQL data types
  2. BOOLEAN
  3. PLS_INTEGER
  4. BINARY_INTEGER
  5. REF CURSOR
  6. User-defined subtypes



SQL data types

- The PL/SQL data types include the SQL data types:
  - Character Data Types
  - Number Data Types
  - Datetime and Interval Data Types
  - RAW and LONG RAW Data Types
  - Large Object (LOB) Data Types
  - Rowid Data Types



Character Data Types

- CHAR Data Type specifies a fixed-length character string in the database character set
- NCHAR data type specifies a fixed-length character string in the national character set
- VARCHAR2 data type specifies a variable-length character string in the database character set.
- NVARCHAR2 data type specifies a variable-length character string in the national character set.

Number Data Types

- The **NUMBER** data type stores zero as well as positive and negative fixed numbers

NUMBER(*p*, *s*)

- *p* is the **precision**: the maximum number of significant decimal digits
- *s* is the **scale**: the number of digits from the decimal point to the least significant digit.

Number Data Types

- The **FLOAT** data type is a subtype of **NUMBER**
  - It can be specified with or without precision
  - Scale cannot be specified, but is interpreted from the data

FLOAT(*p*)

- ***p*** is the **precision**: the maximum number of significant decimal digits

Number Data Types

- Floating-point numbers can have a decimal point anywhere from the first to the last digit or can have no decimal point at all

- Oracle Database provides two numeric data types exclusively for floating-point numbers:
  - **BINARY_FLOAT** is a 32-bit, single-precision floating-point number data type.
    - Each BINARY_FLOAT value requires 4 bytes
  - **BINARY_DOUBLE** is a 64-bit, double-precision floating-point number data type.
    - Each BINARY_DOUBLE value requires 8 bytes.



Datetime and Interval Data Types

- The DATE data type stores date and time information
  - Oracle stores the following information for a DATE: year, month, day, hour, minute, and second

- The TIMESTAMP data type is an extension of the DATE data type
  - It stores all information from of the DATE data type plus fraction of seconds
  - This data type is useful for
    - Storing precise time values
    - Collecting date information across geographic regions
    - Evaluating date information across geographic regions
  - Specify the TIMESTAMP data type as follows:

TIMESTAMP [(*fractional_seconds_precision*)]

- where *fractional_seconds_precision* optionally specifies the number of digits Oracle stores in the fractional part of the SECOND datetime field.



Datetime and Interval Data Types

- TIMESTAMP WITH LOCAL TIME ZONE is another variant of TIMESTAMP that is sensitive to time zone information

TIMESTAMP [(*fractional_seconds_precision*)] WITH LOCAL TIME ZONE

  - where *fractional_seconds_precision* optionally specifies the number of digits Oracle stores in the fractional part of the SECOND datetime field.

Datetime and Interval Data Types

- INTERVAL YEAR TO MONTH stores a period of time using the YEAR and MONTH datetime fields
  - This data type is useful for representing the difference between two datetime values when only the year and month values are significant.

    INTERVAL YEAR [(*year_precision*)] TO MONTH

  - where *year_precision* is the number of digits in the YEAR datetime field

Datetime and Interval Data Types

- INTERVAL DAY TO SECOND stores a period of time in terms of days, hours, minutes, and seconds.
  - This data type is useful for representing the precise difference between two datetime values

INTERVAL DAY [(*day_precision*)] TO SECOND [(*fractional_seconds_precision*)]

  - Where
    - *day_precision* is the number of digits in the DAY datetime field. Accepted values are 0 to 9. The default is 2.
    - *fractional_seconds_precision* is the number of digits in the fractional part of the SECOND datetime field. Accepted values are 0 to 9. The default is 6.



RAW and LONG RAW Data Types

- The RAW and LONG RAW data types store data that is not to be explicitly converted by Oracle Database when moving data between different systems
- These data types are intended for binary data or byte strings
- Oracle strongly recommends that you convert LONG RAW columns to binary LOB (BLOB) columns.
- LOB columns are subject to far fewer restrictions than RAW columns.



Large Object (LOB) Data Types

- The built-in LOB data types
  - BLOB
  - CLOB
  - NCLOB (stored internally)
  - BFILE (stored externally)

- LOBs can store large and unstructured data such as text, image, video, and spatial data.



BFILE Data Types

- The BFILE data type enables access to binary file LOBs that are stored in file systems outside Oracle Database
- A BFILE column or attribute stores a BFILE locator
- BFILE locator serves as a pointer to a binary file on the server file system
- The locator maintains the directory name and the filename
- Binary file LOBs do not participate in transactions and are not recoverable
- The operating system provides file integrity and durability



BFILE Data Types

- BFILE data can be up to 2^64-1 bytes (the operating system may impose restrictions on this maximum)
- The database administrator must ensure that
  - The external file exists
  - Oracle processes have operating system read permissions on the file.
- The BFILE data type enables read-only support of large binary files:
  - Such a file cannot be modified or replicated



BLOB Data Types

- The BLOB data type stores unstructured binary large objects

- BLOB objects can
  - Be thought of as bitstreams with no character set semantics
  - Store binary data up to (4 gigabytes -1) * (the value of the CHUNK parameter of LOB storage)
  - CHUNK is equal to the database block size



CLOB Data Type

- The CLOB data type stores
  - Single-byte character data
  - Multibyte character data
- CLOB supports
  - Fixed-width character sets
  - Variable-width character sets
  - Both use the database character set
- CLOB objects can store up to (4 gigabytes -1) * (the value of the CHUNK parameter of LOB storage) of character data



NCLOB Data Types

- The NCLOB data type stores Unicode data.
- NCLOB supports
  - Fixed-width character sets
  - Variable-width character sets
  - Both use the national character set
- NCLOB objects can store up to (4 gigabytes -1) * (the value of the CHUNK parameter of LOB storage) of character text data



Rowid Data Types

- Each row in the database has an address
- The rows in heap-organized tables that are native to Oracle Database have row addresses called **rowids**
- A rowid row address can be examined by querying the pseudocolumn ROWID.
- Values of this pseudocolumn are strings representing the address of each row.
- These strings have the data type ROWID

Rowid Data Types

- Rowids contain the following information:
  - The **data block** of the data file containing the row.
  - The **row** in the data block.
  - The **database file** containing the row
  - The **data object number** which is an identification number assigned to every database segment.



BOOLEAN Data Type

- The PL/SQL data type BOOLEAN stores **logical values:**
  - TRUE
  - FALSE
  - NULL (representing an unknown value)

- **Because SQL has no data type equivalent to BOOLEAN, it is not possible to:**
  - Assign a BOOLEAN value to a database table column
  - Select or fetch the value of a database table column into a BOOLEAN variable
  - Use a BOOLEAN value in a SQL function
  - Use a BOOLEAN expression in a SQL statement, except
    - As an argument to a PL/SQL function invoked in a SQL query
    - In a PL/SQL anonymous block.



PLS_INTEGER and BINARY_INTEGER Data Types

- The PL/SQL data types PLS_INTEGER and BINARY_INTEGER are identical.
- Both data types
  - Store signed integers in the range -2,147,483,648 through 2,147,483,647
  - Integers are represented in 32 bits
- These data types have the following advantages over the NUMBER data type and NUMBER subtypes:
  - Values require less storage
  - Operations use hardware arithmetic
  - Are faster than NUMBER operations which use library arithmetic



User-Defined PL/SQL Subtypes

- Developers can define their own subtypes for restrincting
  - Size
  - Precision
  - Scale

SUBTYPE *subtype_name* IS *base_type*
{ *precision* [, *scale*] | RANGE *low_value* .. *high_value* } [NOT NULL]



Overview

- Variables and constants
- Data types
- Operators
- Control statements



Operators

- PL/SQL supports:
  - Arithmetic operators
  - Logic operators
  - Comparison operators
  - SQL operators



Arithmetic operators

Operator	Operation	Example
**	Exponential	val2 ** val1
+	Identity	+ val
-	Negation	- val
+	Addition	val1 + val2
-	Subtraction	val1 - val2
*	Multiplication	val1 * val2
/	Division	val1 / val2
||	Concatenation	val1 || val2



Logical operators

Operator	Operation	Example
NOT	Negation	NOT val
AND	Logical AND	val1 AND val2
OR	Logical OR	val1 OR val2



Comparison operators

Operator	Operation	Example
=	Equal	val2 = val1
<	Less then	val2 < val1
>	Greater than	val2 > val1
<=	Less or equal than	val1 <= val2
>=	Greater or equal than	val1 >= val2
<>	Different than	val1 <> val2
!=	Different than	val1 != val2
~=	Different than	val1 ~= val2
^=	Different than	val1 ^= val2

All comparison operators can be negates using **NOT**



SQL operators

Operator	Operation	Example
BETWEEN	Verifies if a value is in a range	val BETWEEN val1 AND val2
IN	Verifies if a value is in a list	val IN(val1, val2, …, val3)
LIKE	Verifies a specified pattern	val LIKE pattern
IS NULL	Verifies if a value is NULL	val IS NULL



SQL operators

- All SQL operators can be negates using **NOT**

- There are two wildcards often used in conjunction with the **LIKE** operator:
  - **%** - The percent sign represents zero, one, or multiple characters
  - _ - The underscore represents a single character



Operators precedence

Precedence	Operators	Operation
1	**	Exponential
2	+, -	Identity, negation
3	*, /	Multiplication, division
4	+, -, ||	Addition, subtraction, concatenation
5	=, <, >, <=, >=, <>, !=, ~=, ^=, IS NULL, LIKE, BETWEEN, IN.	Comparison and SQL operators
6	NOT	Logical negation
7	AND	Conjunction
8	OR	Inclusion



Other symbols

Symbol	Meaning
:=	Assignment operator
=>	Association operator
%	Attribute indicator
'	String delimiter
.	Component selector
( )	Expression or list delimiter
:	Host variable indicator
,	Item separator
<< >>	Label delimiter
/* */	Multiline comment delimiter
"	Quoted identifier delimiter
..	Range operator
@	Remote access indicator
--	Single-line comment indicator
;	Statement terminator



Overview

- Variables and constants
- Data types
- Operators
- Control statements



Control Statements

- Conditional selection statements
- Loop statements
- Sequential control statements



Conditional selection statements
IF Statement

```
1 -- IF THEN Statement
2 IF condition THEN statements
3 END IF;
4
5 -- IF THEN ELSE Statement
6 IF condition THEN statements
7 ELSE else_statements
8 END IF;
9
10 -- IF THEN ELSIF Statement
11 IF condition_1 THEN statements_1
12 ELSIF condition_2 THEN statements_2
13 [ ELSIF condition_3 THEN statements_3 ]
14 ...
15 [ ELSE else_statements ]
16 END IF;
17
18 -- Logically equivalent nested
19 -- IF THEN ELSE statements
20 IF condition_1 THEN statements_1;
21 ELSE
22   IF condition_2 THEN statements_2;
23   ELSE
24     IF condition_3 THEN statements_3;
25     ...
26     END IF;
27   END IF;
28 END IF;
```

- The condition can contain
  - Comparison operators
  - Logical operators
  - SQL operators



Conditional selection statements CASE Statement

- Simple CASE
  - The selector is an expression (typically a single variable)
  - Each selector_value can be either a literal or an expression (cannot use NULL)
  - The simple CASE statement runs the first statements for which selector_value equals selector
  - Remaining conditions are not evaluated
  - If no selector_value equals selector
    - The CASE statement runs else_statements if they exist
    - Raises the predefined exception CASE_NOT_FOUND otherwise
- Searched CASE
  - The searched CASE statement runs the first statements for which condition is true
  - Remaining conditions are not evaluated

```
1  -- Simple CASE
2  CASE selector
3  WHEN selector_value_1 THEN statements_1
4  WHEN selector_value_2 THEN statements_2
5  ...
6  WHEN selector_value_n THEN statements_n
7  [ ELSE else_statements ]
8  END CASE;
9
10 -- Searched CASE
11 CASE
12 WHEN condition_1 THEN statements_1
13 WHEN condition_2 THEN statements_2
14 ...
15 WHEN condition_n THEN statements_n
16 [ ELSE else_statements ]
17 END CASE;
```



Conditional selection statements Basic LOOP Statement

- With each iteration of the loop:
  - The statements are run
  - Control returns to the top of the loop
- To prevent an infinite loop and exit the loop
  - Use a statement
  - Raised exception

```
1 [label] LOOP
2   statements
3   [ EXIT [ WHEN condition ] ]
4   [ CONTINUE [ WHEN condition ] ]
5 END LOOP [label];
```



Conditional selection statements Basic LOOP Statement

- The EXIT statement
  - Exits the current iteration of a loop unconditionally
  - Transfers control to the end of either
    - The current loop
    - An enclosing labeled loop



Conditional selection statements Basic LOOP Statement

- The EXIT WHEN statement
  - Exits the current iteration of a loop when the condition in its WHEN clause is true
  - Transfers control to the end of either
    - The current loop
    - An enclosing labeled loop
  - The condition in the WHEN clause is evaluated each time control reaches the EXIT WHEN statement
  - The EXIT WHEN statement does nothing if the condition is not true
  - A statement inside the loop must make the condition true to prevent an infinite loop



Conditional selection statements Basic LOOP Statement

- The CONTINUE statement
  - Continues the current iteration of a loop unconditionally
  - Transfers control to the next iteration of either
    - The current loop
    - An enclosing labeled loop



Conditional selection statements Basic LOOP Statement

- The CONTINUE WHEN statement
  - Continues the current iteration of a loop when the condition in its WHEN clause is true
  - Transfers control to the next iteration of either
    - The current loop
    - An enclosing labeled loop
  - The condition in the WHEN clause is evaluated each time control reaches the CONTINUE WHEN statement
  - The CONTINUE WHEN statement does nothing if the condition is not true



Conditional selection statements WHILE LOOP Statement

- The WHILE LOOP statement runs one or more statements while a condition is true

```
1 [label] WHILE condition LOOP
2   statements
3   [ EXIT [ WHEN condition ] ]
4   [ CONTINUE [ WHEN condition ] ]
5 END LOOP [label];
```

Conditional selection statements
WHILE LOOP Statement

- If the condition is true
  - Statements are run
  - Control returns to the top of the loop where condition is evaluated again
- The WHILE LOOP stops when the condition is false
- A statement inside the loop must make the condition false or null to prevent an infinite loop
- An EXIT, EXIT WHEN, CONTINUE, or CONTINUE WHEN in the statements can cause the loop or the current iteration of the loop to end early

```
1 [label] WHILE condition LOOP
2   statements
3   [ EXIT [ WHEN condition ] ]
4   [ CONTINUE [ WHEN condition ] ]
5 END LOOP [label];
```



Conditional selection statements FOR LOOP Statement

- The FOR LOOP statement runs one or more statements while the loop index is in a specified range

```
1 [label] FOR IN [ REVERSE ] LOWER_BOUND .. UPPER_BOUND
2 LOOP
3   statements
4   [ EXIT [ WHEN condition ] ]
5   [ CONTINUE [ WHEN condition ] ]
6 END LOOP [label];
7
```



Conditional selection statements FOR LOOP Statement

- Without REVERSE:
  - The value of index starts at lower_bound and increases by one with each iteration of the loop until it reaches upper_bound
  - The statements never run if lower_bound is greater than upper_bound
- Without REVERSE:
  - The value of index starts at upper_bound and decreases by one with each iteration of the loop until it reaches lower_bound
  - If upper_bound is less than lower_bound, then the statements never run
- An EXIT, EXIT WHEN, CONTINUE, or CONTINUE WHEN in the statements can cause the loop or the current iteration of the loop to end early

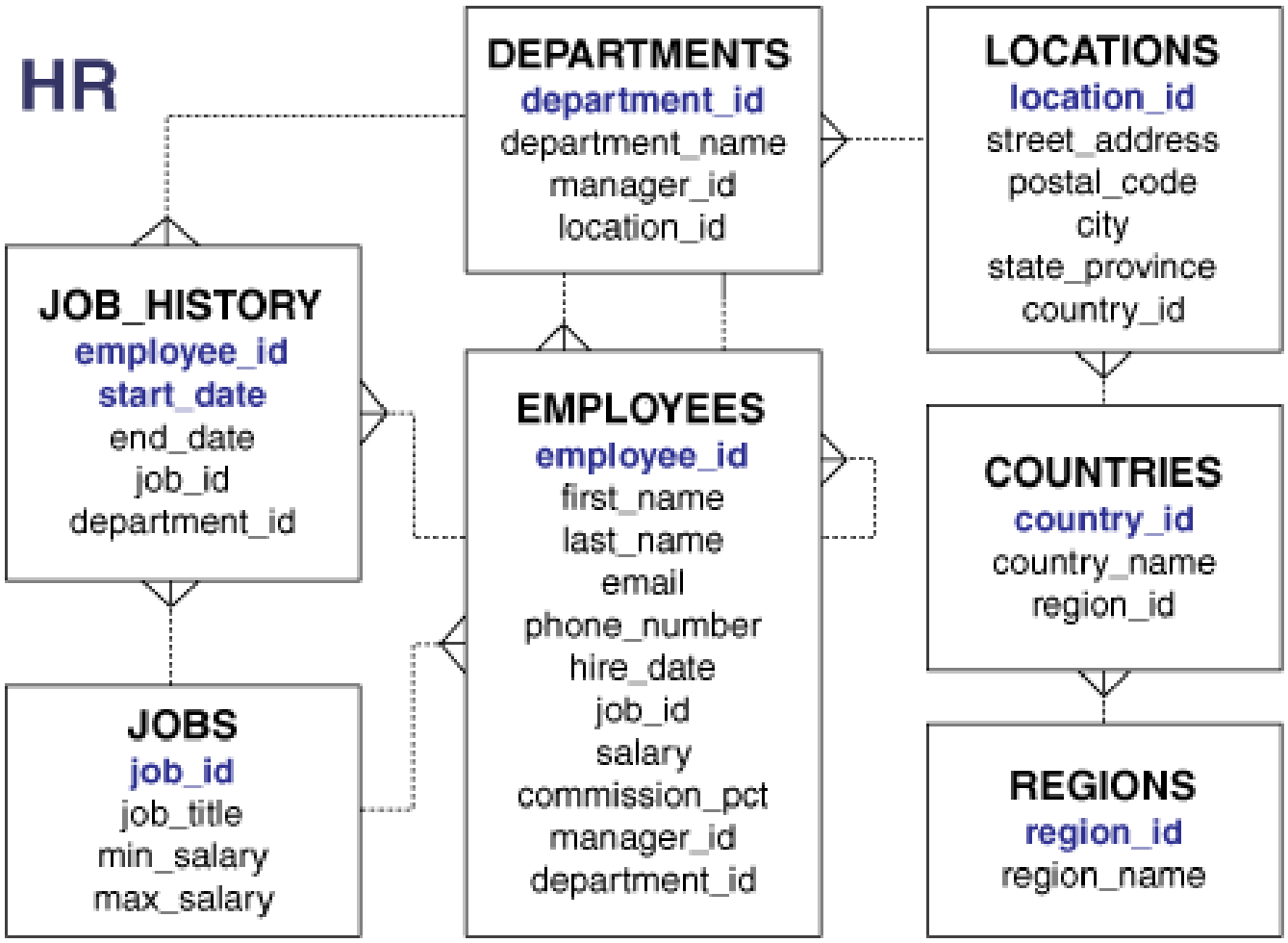
```
1 -- FOR LOOP
2 FOR IN [ REVERSE ] LOWER_BOUND .. UPPER_BOUND
3 LOOP
4   statements
5   [ EXIT [ WHEN condition ] ]
6   [ CONTINUE [ WHEN condition ] ]
7 END LOOP;
```



Conditional selection statements Sequential Control Statements

- The sequential control statements:
  - GOTO
    - Transfers control to a label unconditionally
    - Can be used to exit a LOOP
  - NULL statement only passes control to the next statement

Examples


HR
DEPARTMENTS
department_id
department_name
manager_id
location_id
LOCATIONS
location_id
street_address
postal_code
city
state_province
country_id
JOB_HISTORY
employee_id
start_date
end_date
job_id
department_id
EMPLOYEES
employee_id
first_name
last_name
email
phone_number
hire_date
job_id
salary
commission_pct
manager_id
department_id
COUNTRIES
country_id
country_name
region_id
JOBS
job_id
job_title
min_salary
max_salary
REGIONS
region_id
region_name



Examples

```
 1 DECLARE
 2     no_months constant BINARY_INTEGER := 12;
 3     emp_name varchar2(45);
 4     emp_salary employees.salary%TYPE;
 5     anual_salary float(10);
 6 BEGIN
 7     SELECT first_name||' '||last_name, salary
 8     INTO emp_name, emp_salary
 9     FROM employees  WHERE employee_id = &id;
10
11     anual_salary := emp_salary * no_months;
12     dbms_output.put_line(rpad(emp_name, 20)
13         ||'-'||lpad(anual_salary,10));
14 END;
15 /
```



Examples

```
DECLARE
    emp_name varchar2(45);
    emp_salary employees.salary%TYPE;
    emp_iddept employees.department_id%TYPE;
    avg_sal number(6,2);
    avg_sal_dept float(8);
BEGIN
    SELECT first_name||' '||last_name, salary, department_id
    INTO emp_name, emp_salary, emp_iddept
    FROM employees WHERE employee_id = &id;

    SELECT AVG(salary) INTO avg_sal FROM employees;

    SELECT AVG(salary) INTO avg_sal_dept FROM employees
    WHERE department_id= emp_iddept;

    IF emp_salary > avg_sal THEN
        dbms_output.put_line(emp_name ||
            ' are sal > decat sal mediu din firma');
    END IF;

    IF emp_salary > avg_sal THEN
        dbms_output.put_line(emp_name ||
        ' are sal > sal mediu din departamentul in care lucreaza');
    ELSE
        dbms_output.put_line(emp_name ||
        ' are sal <= sal mediu din departamentul in care lucreaza');
    END IF;
END;
/
```



Examples

```
DECLARE
    year_of_experience INTERVAL YEAR TO MONTH;
    expert INTERVAL YEAR TO MONTH;
    emp_name varchar2(45);
    emp_hiredate employees.hire_date%TYPE;
BEGIN
    expert := INTERVAL '15-1' YEAR TO MONTH;

    SELECT first_name||' '||last_name, hire_date
    INTO emp_name, emp_hiredate
    FROM employees WHERE employee_id = &id;

    IF emp_hiredate < sysdate - expert THEN
        dbms_output.put_line(emp_name||' e expert');
    ELSE
        dbms_output.put_line(emp_name||' nu e expert');
    END IF;
END;
/
```



Examples

```
 1 DECLARE
 2     emp_name varchar2(45);
 3     comm employees.commission_pct%TYPE;
 4 BEGIN
 5     SELECT first_name||' '||last_name, commission_pct
 6     INTO emp_name, comm
 7     FROM employees WHERE employee_id = &id;
 8     CASE
 9         WHEN comm IS NULL THEN
10             dbms_output.put_line(emp_name || ' nu are commision');
11         WHEN comm >= 0 and comm < 0.2 THEN
12             dbms_output.put_line(emp_name || ' are commision mic');
13         WHEN comm >= 0.2 and comm < 0.3 THEN
14             dbms_output.put_line(emp_name || ' are commision mediu');
15         ELSE
16             dbms_output.put_line(emp_name || ' are commision mare');
17     END CASE;
18 END;
19 /
```



Examples

```
DECLARE
    emp_name varchar2(45);
    comm employees.commission_pct%TYPE;
BEGIN
    SELECT first_name||' '||last_name, commission_pct
    INTO emp_name, comm
    FROM employees WHERE employee_id = &id;

    IF comm IS NULL THEN
        dbms_output.put_line(emp_name || ' nu are commision');
    ELSIF comm >= 0 and comm < 0.2 THEN
        dbms_output.put_line(emp_name || ' are commision mic');
    ELSIF comm >= 0.2 and comm < 0.3 THEN
        dbms_output.put_line(emp_name || ' are commision mediu');
    ELSE
        dbms_output.put_line(emp_name || ' are commision mare');
    END IF;
END;
/
```



Examples

```
DECLARE
    emp_name varchar2(45);
    emp_iddept employees.department_id%TYPE;
    emp_reg regions.region_name%TYPE;
    emp_city locations.city%TYPE;
BEGIN
    SELECT first_name||' '||last_name, department_id
    INTO emp_name, emp_iddept
    FROM employees WHERE employee_id = &id;

    SELECT r.region_name, l.city INTO emp_reg, emp_city FROM regions r
    INNER JOIN countries c ON c.region_id = r.region_id
    INNER JOIN locations l ON l.country_id = c.country_id
    INNER JOIN departments d ON d.location_id = l.location_id
    WHERE d.department_id = emp_iddept;

    CASE emp_reg
        WHEN 'Europe' THEN
            IF emp_city = 'London' THEN
                dbms_output.put_line(emp_name||' works in European HQ');
            ELSE
                dbms_output.put_line(emp_name||' doesn''t works in HQ');
            END IF;
        WHEN 'Americas' THEN
            IF emp_city = 'Seattle' THEN
                dbms_output.put_line(emp_name||' works in American HQ');
            ELSE
                dbms_output.put_line(emp_name||' doesn''t works in HQ');
            END IF;
        ELSE
            dbms_output.put_line(emp_name||' works outside Europe and America');
    END CASE;
END;
/
```



Examples

```
 1 DECLARE
 2     idx number := 1;
 3     deptname departments.department_name%TYPE;
 4     avg_sal_dept number(7,2);
 5 BEGIN
 6     LOOP
 7         SELECT d.department_name, avg(e.salary)
 8         INTO deptname, avg_sal_dept
 9         FROM departments d
10         INNER JOIN employees e
11         ON d.department_id = e.department_id
12         WHERE d.department_id = idx * 10
13         GROUP BY d.department_name;
14
15         dbms_output.put_line(deptname||' '||avg_sal_dept);
16         idx := idx + 1;
17
18         CONTINUE WHEN idx < 12;
19
20         EXIT WHEN idx = 12;
21
22     END LOOP;
23 END;
24 /
```



Examples

```
DECLARE
    idx number := 1;
    deptname departments.department_name%TYPE;
    avg_sal_dept number(7,2);
BEGIN
    WHILE idx < 12
    LOOP
        SELECT d.department_name, avg(e.salary)
        INTO deptname, avg_sal_dept
        FROM departments d
        INNER JOIN employees e
        ON d.department_id = e.department_id
        WHERE d.department_id = idx * 10
        GROUP BY d.department_name;

        dbms_output.put_line(deptname||' '||avg_sal_dept);
        idx := idx + 1;
    END LOOP;
END;
/
```



Examples

```
DECLARE
    deptname departments.department_name%TYPE;
    avg_sal_dept number(7,2);
BEGIN
    FOR idx IN 1 .. 12
    LOOP
        IF idx = 12 THEN
            GOTO exit_for;
        END IF;

        SELECT d.department_name, avg(e.salary)
        INTO deptname, avg_sal_dept
        FROM departments d
        INNER JOIN employees e
        ON d.department_id = e.department_id
        WHERE d.department_id = idx * 10
        GROUP BY d.department_name;

        dbms_output.put_line(deptname||' '||avg_sal_dept);
    END LOOP;

    <<exit_for>>dbms_output.put_line('Done!');
END;
/
```